CHAPTER 22

Support Vector Machines

Support vector machine (SVM) is a machine learning algorithm for learning classification and regression models. To build intuition, we will consider the case of learning a classification model with SVM. Given a dataset with two target classes that are linearly separable, it turns out that there exists an infinite number of lines that can discriminate between the two classes (see Figure 22-1). The goal of the SVM is to find the best line that separates the two classes. In higher dimensions, this line is called a hyperplane.

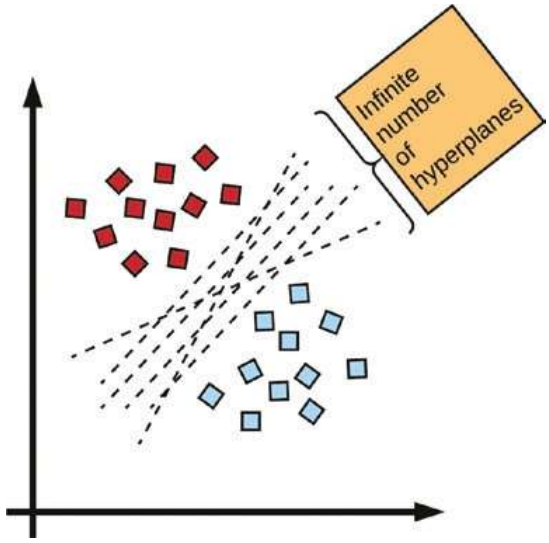

Figure 22-1. *Infinite set of discriminants*

What Is a Hyperplane?

A hyperplane is a line or more technically called a discriminant that separates two classes in n-dimensional space. When a hyperplane is drawn in 2-D space, it is called a line. In 3-D space, it is called a plane, and in dimensions greater than 3, the discriminant is called a hyperplane (see Figure 22-2). For any n-dimensional world, we have n-1 hyperplanes.

E. Bisong, *Building Machine Learning and Deep Learning Models on Google Cloud Platform*,
https://doi.org/10.1007/978-1-4842-4470-8_22

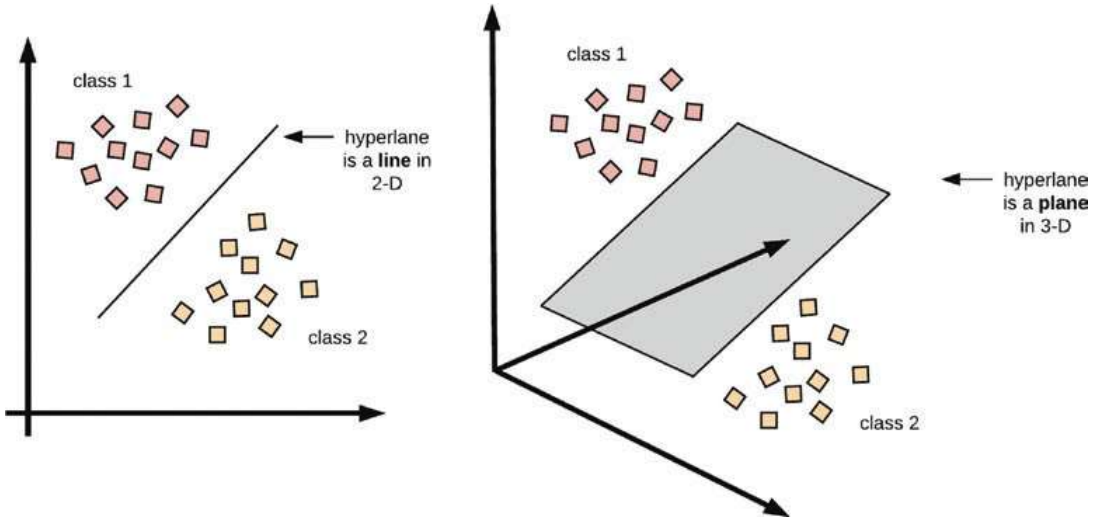

Figure 22-2. *Left: A hyperplane in 2-D is a line. Right: A hyperplane in 3-D is a plane. For dimension greater than 3, visualization becomes difficult.*

Finding the Optimal Hyperplane

The best hyperplane that linearly separates two classes is identified as the line lying at the largest margin from the nearest vectors at the boundary of the two classes.

In Figure 22-3, we observe that the best hyperplane is the line at the exact center of the two classes and constitutes the largest margin between both classes. Hence, this optimal hyperplane is also known as the largest margin classifier.

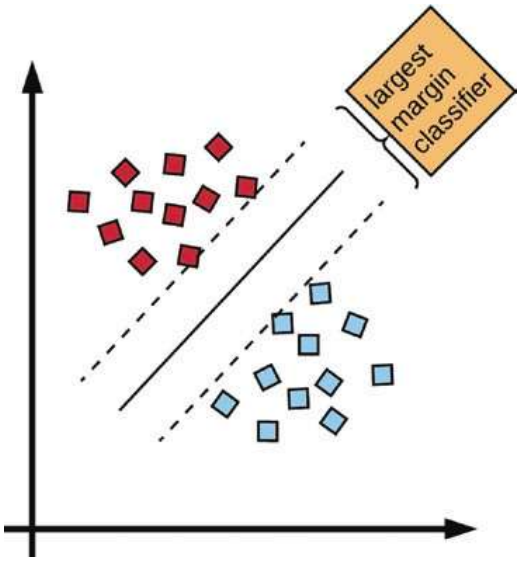

Figure 22-3. *The largest margin classifier*

The boundary points of the respective classes which are known as the support vectors are essential in finding the optimal hyperplane. The support vectors are illustrated in Figure 22-4. The boundary points are called support vectors because they are used to determine the maximum distance between the class they belong to and the discriminant function separating the classes.

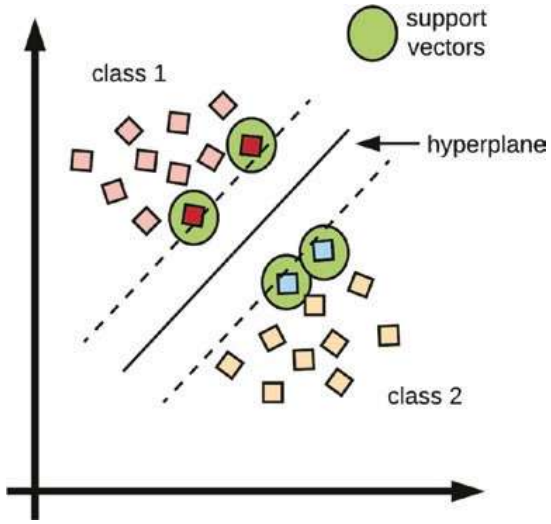

Figure 22-4. *Support vectors*

The mathematical formulation for finding the margin and consequently the hyperplane that maximizes the margin is beyond the scope of this book, but suffice to say this technique involves the Lagrange multiplier.

The Support Vector Classifier

In the real world, it is difficult to find data points that are precisely linearly separable and for which exists a large margin hyperplane. In Figure 22-5, the left image represents the data points for two classes in a dataset. Observe that there readily exists a linear separator between those two classes. Now, suppose we have an additional point from class 1 adjusted in such a way that it is much closer to class 2, we see that this point upsets the location of the hyperplane as seen in the right image of Figure 22-5. This reveals the sensitivity of the hyperplane to an additional data point that may result in a very narrow margin.

This sensitivity to data samples has significant drawbacks, the first being that the distance between the support vectors and the hyperplane reflects the confidence in the classification accuracy. Also, the drastic change in the position of the hyperplane due to a single additional point shows that the classifier is susceptible to high variability and can overfit the training data.

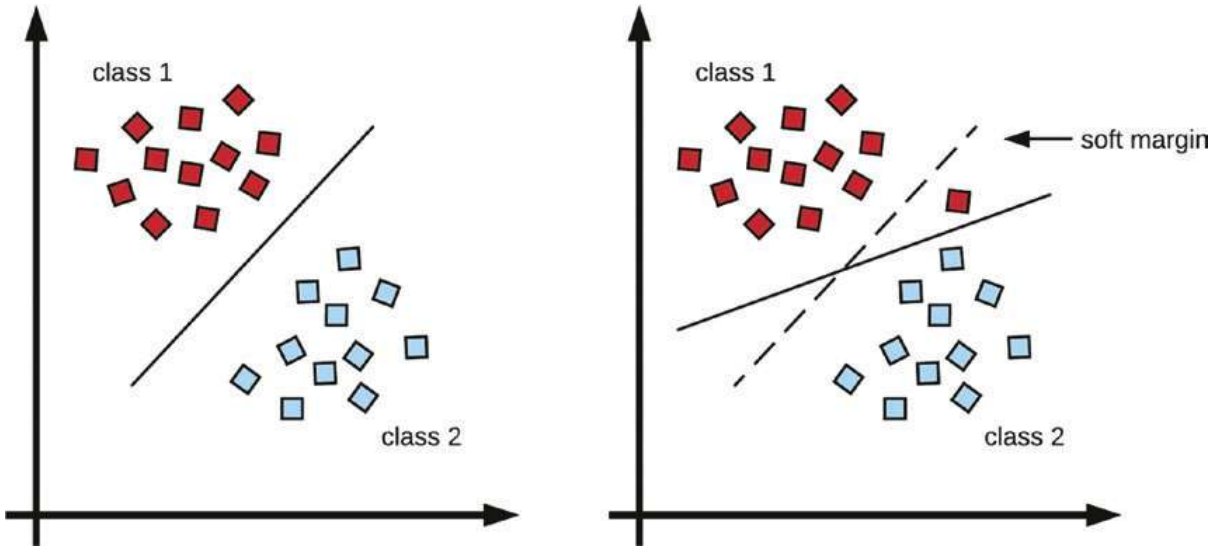

Figure 22-5. *Left: A linearly separable data distribution with a large margin. Right: The data point distribution makes it more difficult to find a large margin classifier that linearly separates the two classes*

The goal of the support vector classifier is to find a hyperplane that nearly discriminates between the two classes. This technique is also called a soft margin. A soft margin is tuned to ignore a degree of error when finding the separating hyperplane. This concept of a soft margin is how we generalize the support vector classifier to find a hyperplane in datasets that are not readily linearly separable. The margin is called soft because some examples are purposefully misclassified.

In such cases, as outlined in Figure 22-5, a soft margin classifier is preferred as it is more insensitive to individual data points and overall will have a better chance of generalizing to new examples. Howbeit, this might misclassify a couple of examples while training, but this is overall beneficial to the quality of the classifier as it generalizes to new samples.

Again, the margin is called soft because some examples are allowed to violate the margin or even be misclassified by the hyperplane to preserve overall generalizability. This is illustrated in Figure 22-6.

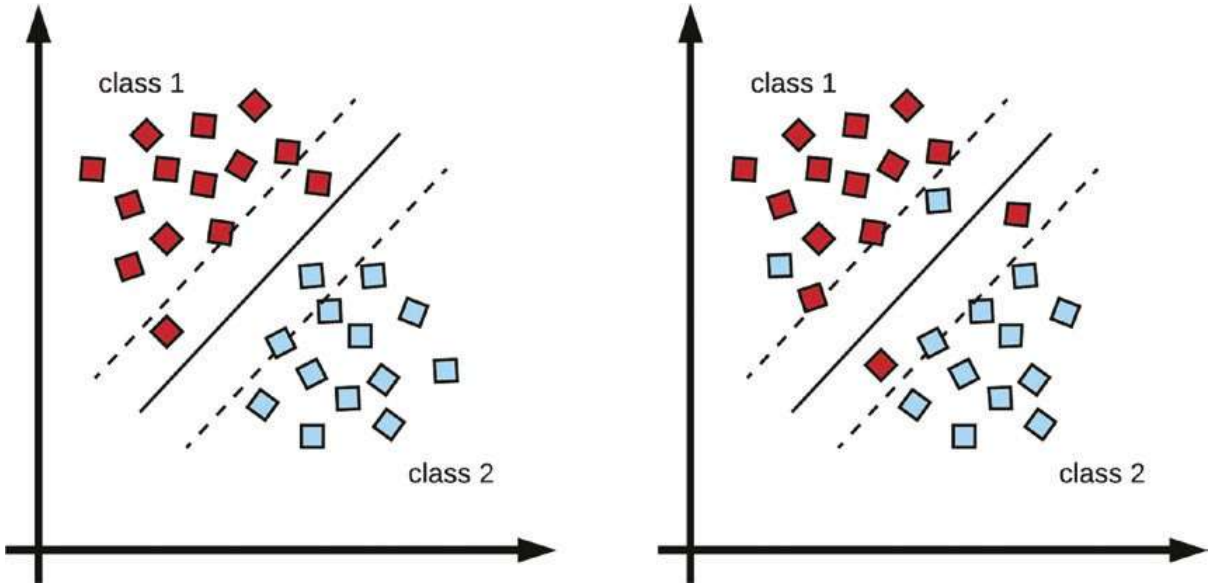

Figure 22-6. *Left: An example of a soft margin with points allowed to violate the margin. Right: An example with some points intentionally misclassified.*

The C Parameter

The C parameter is the hyper-parameter that is responsible for controlling the degree of violations to the margins or the number of intentionally misclassified points allowed by the support vector classifier. The C hyper-parameter is a non-negative real number. When this C parameter is set to 0, the classifier becomes the large margin classifier.

In a soft margin classifier, the C parameter is tuned by adjusting its values to control the tolerance of the margin. With larger values of C, the classifier margins become wider and more tolerant to violations and misclassifications. However, with smaller values of C, the margins become narrower and are less tolerant of violations and misclassified points.

Observe that the C hyper-parameter is vital for regulating the bias/variance trade-off of the support vector classifier. The higher the value of C, our classifier is more prone to variability in the data points and can under-simplify the learning problem. Also, if C is set closer to zero, it results in a much narrower margin, and this can overfit the classifier, leading to high variance – and this will likely fail to generalize to new examples (see Figure 22-7).

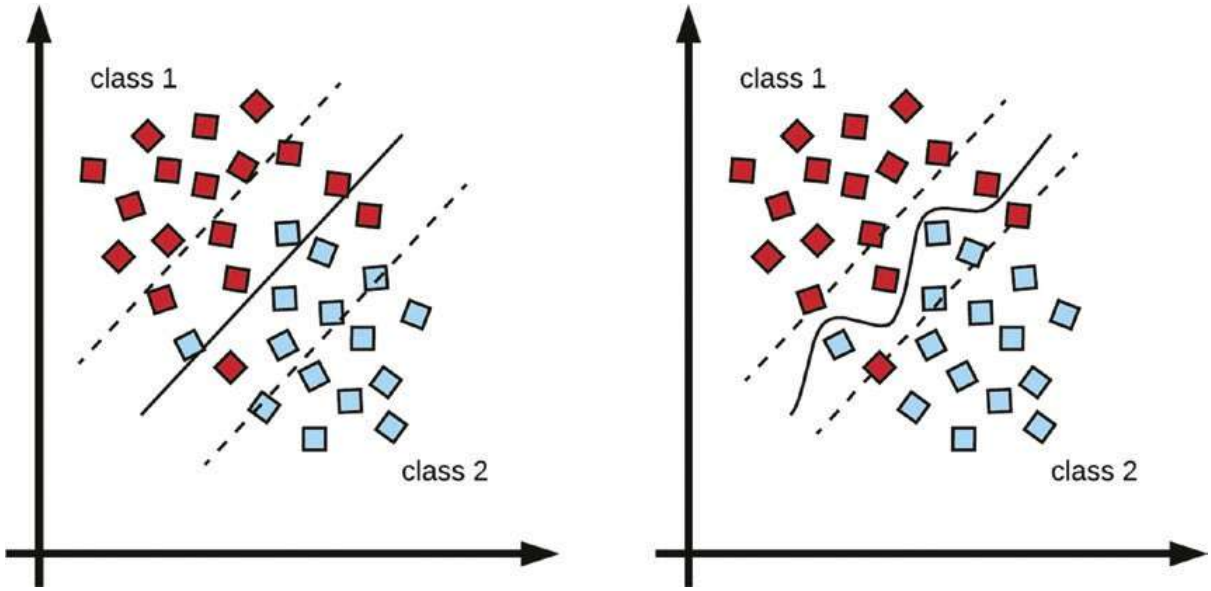

Figure 22-7. *Left: Higher values of C result in wider margins with more tolerance. Right: Lower values of C result in narrower margins with less tolerance*

Multi-class Classification

Previously, we have used the SVC to build a discriminant classifier for binary classes. What happens when we have more than two classes of outputs in the dataset, which is often the case in practice? The SVM can be extended for classifying k classes within a dataset, where k > 2. This extension is, however, not trivial with the SVM. There exist two standard approaches for addressing this problem. The first is the one-vs.-one (OVO) multi-class classification, while the other is the one-vs.-all (OVA) or one-vs.rest (OVR) multi-class classification technique.

One-vs.-One (OVO)

In the one-vs.-one approach, when the number of classes, k, is greater than 2, the algorithm constructs "k combination 2", $\left(\frac{k}{2}\right)$ classifiers, where each classifier is for a pair of classes. So if we have 10 classes in our dataset, a total of 45 classifiers is constructed or trained for every pair of classes. This is illustrated with four classes in Figure 22-8.

After training, the classifiers are evaluated by comparing examples from the test set against each of the $\left(\frac{k}{2}\right)$ classifiers. The predicted class is then determined by choosing the highest number of times an example is assigned to a particular class.

The one-vs.-one multi-class technique can potentially lead to a large number of constructed classifiers and hence can result in slower processing time. Conversely, the classifiers are more robust to class imbalances when training each classifier.

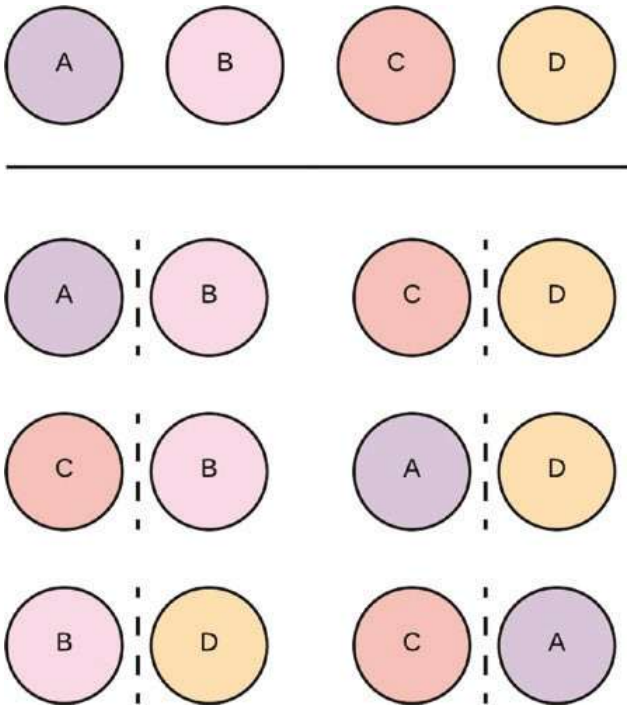

Figure 22-8. *Suppose we have four classes in the dataset labeled A to D, this will result in six different classifiers*

One-vs.-All (OVA)

The one-vs.-all method for fitting an SVM to a multi-classification problem where the number of classes k is greater than 2 consists of fitting each k class against the remaining k – 1 classes. Suppose we have ten classes, each of the classes will be classified against the remaining nine classes. This example is illustrated with four classes in Figure 22-9.

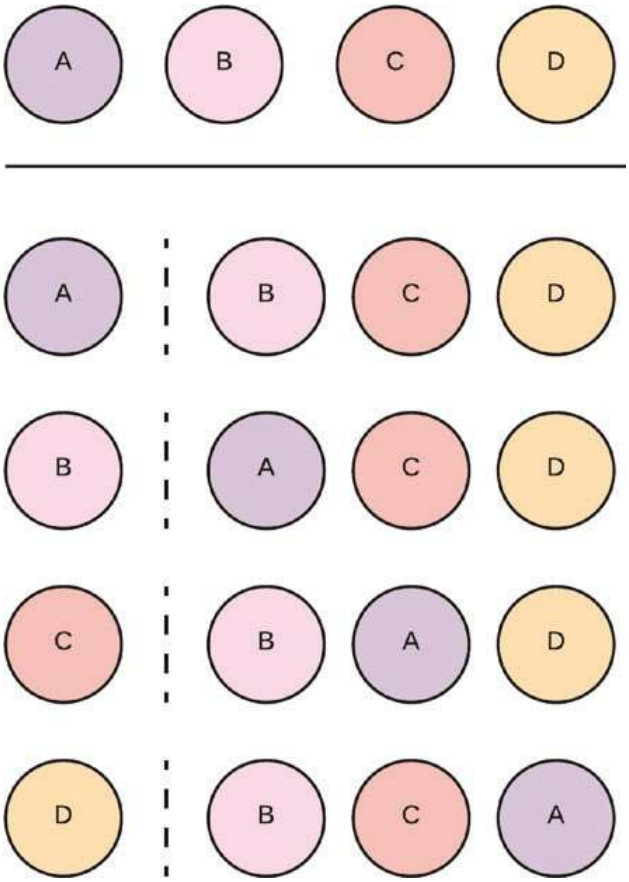

Figure 22-9. *Given four classes in a dataset, we construct four classifiers, with each class fitted against the rest*

The classifiers are evaluated by comparing a test example to each fitted classifier. The classifier for which the margin of the hyperplane is the largest is chosen as the predicted classification target because the classifier margin size is indicative of high confidence of class membership.

The Kernel Trick: Fitting Non-linear Decision Boundaries

Non-linear datasets occur more often than not in real world scenarios.

Technically speaking, the name support vector machine is when a support vector classifier is used with a non-linear kernel to learn non-linear decision boundaries.

SVM uses an essential technique for extending the feature space of a dataset to construct a non-linear classifier. This technique is called kernel and is popularly known as the kernel trick. Figure 22-10 illustrates the kernel trick as an extra dimension is added to the feature space.

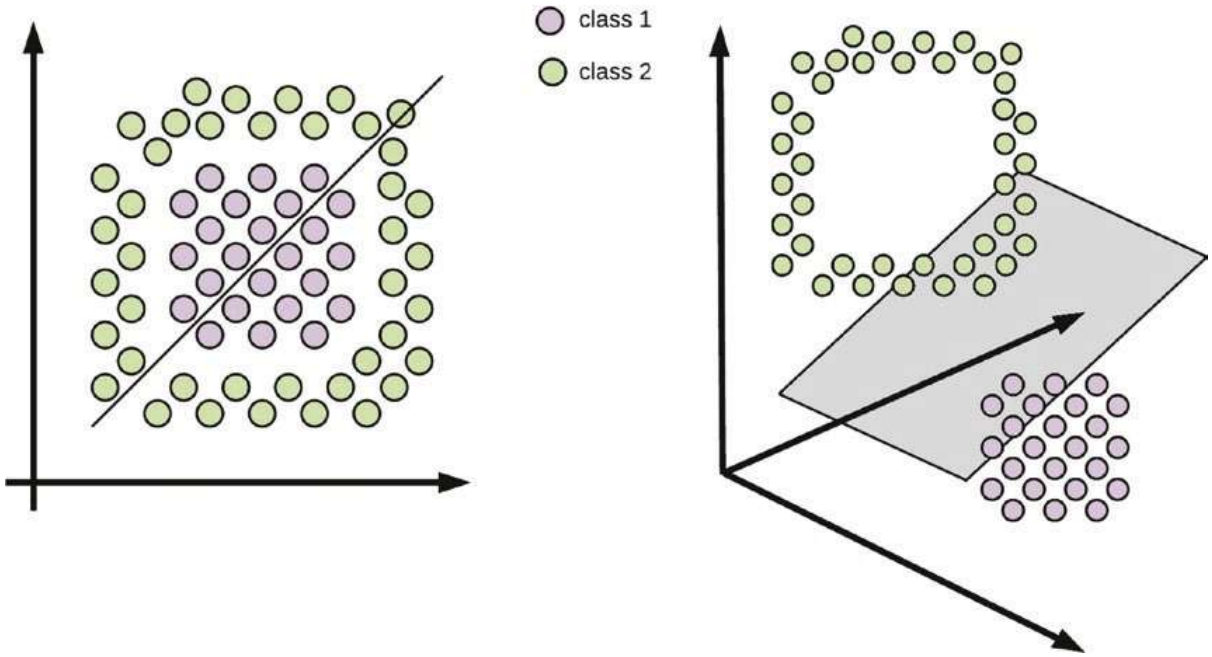

Figure 22-10. *Left: Linear discriminant to non-linear data. Right: By using the kernel trick, we can linearly separate a non-linear dataset by adding an extra dimension to the feature space.*

Adding Polynomial Features

The feature space of the dataset can be extended by adding higher-order polynomial terms or interaction terms. For example, instead of training the classifier with linear features, we can add polynomial features or add interaction terms to our model.

Depending on the dimensions of the dataset, the combinations for extending the feature space can quickly become unmanageable, and this can easily lead to a model that overfits the test set and also become expensive to compute with a larger feature space.

Kernels

Kernel is a mathematical procedure for extending the feature space of a dataset to learn non-linear decision boundaries between different classes. The mathematical details of kernels are beyond the scope of this text. Suffice to say that a kernel can be seen as a mathematical function that captures similarity between data samples.

Linear Kernel

The support vector classifier is the same as a linear kernel. It is also known as a linear kernel because the feature space of the support vector classifier is linear.

Polynomial Kernel

The kernel can also be expressed as a polynomial. With this, a support vector classifier is trained on higher-dimensional polynomial features without manually adding an exponential number of polynomial features to the dataset. Adding a polynomial kernel to the support vector classifier enables the classifier to learn a non-linear decision boundary.

Radial Basis Function or the Radial Kernel

The radial basis function or radial kernel is another non-linear kernel that enables the support vector classifier to learn a non-linear decision boundary. The radial kernel is similar to adding multiple similarity features to the space. For the radial basis function, a hyper-parameter called gamma, γ, is used to control the flexibility of the non-linear decision boundary. The smaller the gamma value, the less complex (or flexible) the non-linear discriminant becomes, but a larger value for gamma leads to a more flexible and sophisticated decision boundary that tightly fits the non-linearity in the data, which can inadvertently lead to overfitting. This is illustrated in Figure 22-11. RBF is a popular kernel option used in practice.

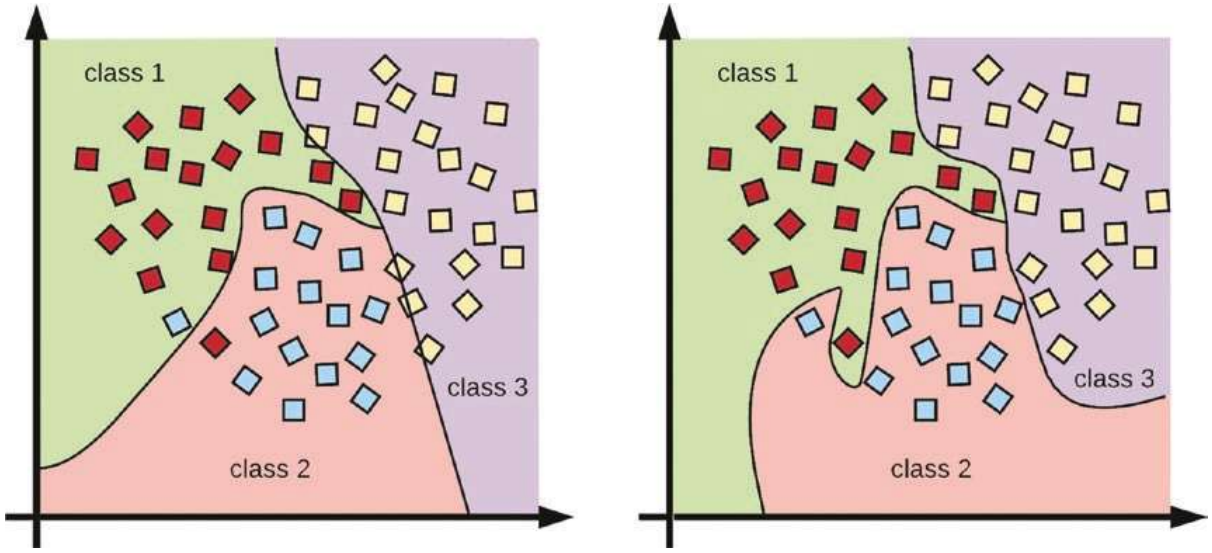

Figure 22-11. *An illustration of adjusting the radial basis function γ parameter, together with the C parameter of the support vector classifier to fit a non-linear decision boundary. Left: RBF kernel with C = 1 and $\gamma = 10^{-3}$. Right: RBF kernel with C = 1 and $\gamma = 10^{-5}$.*

When using the radial kernel with the support vector classifier, the values of C and gamma are hyper-parameters that are tuned to find an appropriate level of model flexibility that generalizes to new examples when deployed.

In practice, a linear kernel or support vector classifier sometimes surprisingly performs well when used to map a function to non-linear data. This observation follows Occam's razor which suggests that it is advantageous to select the simplest hypothesis to solve a problem in the presence of more complex options.

Also, with regard to choosing the best set of C and gamma, γ, to avoid overfitting, a grid search is used to explore a range of values for the hyper-parameters and come up with the combination that performs best on test data. The grid search is used in conjunction with cross-validation approaches. However, the grid search procedure can be potentially computationally expensive.

Support vector machines perform well with high-dimensional data. However, they are preferred for small or medium-sized datasets. For humongous datasets, SVMs become computationally infeasible. Another limitation is that the performance of SVMs is known to plateau at some point, even when there exist large training samples. This is one of the motivations and advantages of deep neural networks.

Support Vector Machines with Scikit-learn

In Scikit-learn, **SVC** is the SVM package for classification, while **SVR** is the SVM package for regression. The attribute 'gamma' in both the SVC and SVR methods controls the flexibility of the decision boundary, and the default kernel is the radial basis function (rbf).

SVM for Classification

In this code example, we will build an SVM classification model to predict the three species of flowers from the Iris dataset.

```
# import packages
from sklearn.svm import SVC
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from math import sqrt

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
svc_model = SVC(gamma='scale')

# fit the model on the training set
svc_model.fit(X_train, y_train)

# make predictions on the test set
predictions = svc_model.predict(X_test)
```

```
# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))
```

```
'Output':
Accuracy: 0.95
```

SVM for Regression

In this code example, we will build an SVM regression model to predict house prices from the Boston house-prices dataset.

```
# import packages
from sklearn.svm import SVR
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
svr_model = SVR(gamma='scale')

# fit the model on the training set
svr_model.fit(X_train, y_train)

# make predictions on the test set
predictions = svr_model.predict(X_test)

# evaluate the model performance using the root mean squared error metric
print("Mean squared error: %.2f" % sqrt(mean_squared_error(y_test,
predictions)))
```

```
'Output':
Root mean squared error: 7.58
```

In this chapter, we surveyed the support vector machine algorithm and its implementation with Scikit-learn. In the next chapter, we will discuss on ensemble methods that combine outputs of multiple classifiers or weak learners to build better prediction models.